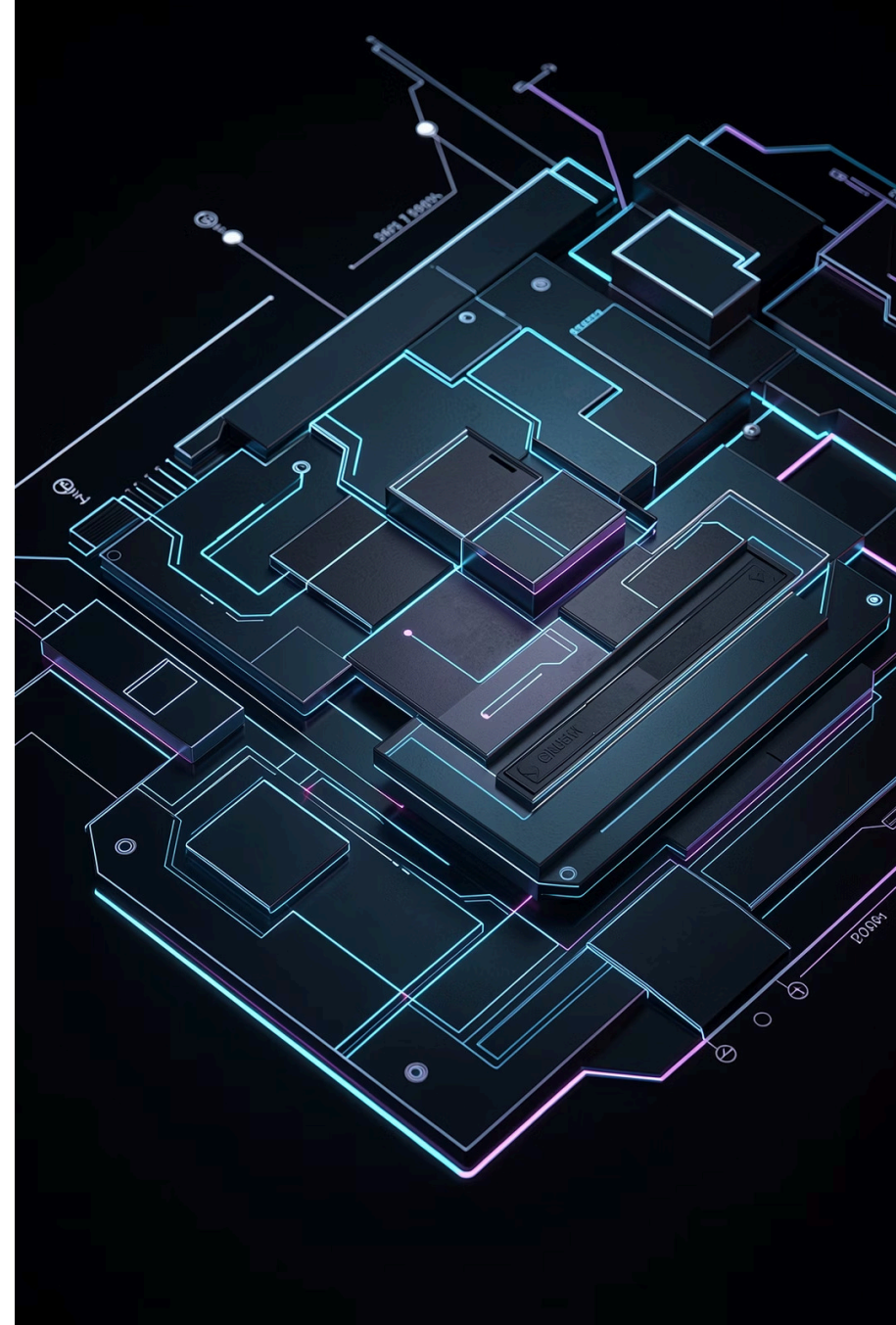


Classes & Objects

A comprehensive guide to Object-Oriented Programming — from core concepts and real-world use cases to hands-on labs, design patterns, and best practices.

`</>` OOP FUNDAMENTALS

BEGINNER TO ADVANCED



Topic Classification

Category

Object-Oriented
Programming (OOP) /
Programming Paradigms

Domain

Software Engineering,
System Design, Application
Development

Difficulty

Beginner to Advanced
(progressive)

Prerequisites

Variables, functions, control
flow, arrays

Recommended Learning Path

01

Procedural Review

Revisit variables, functions, and
control flow

02

Classes & Objects Basics

Blueprints, instances, attributes,
methods

03

Core OOP Pillars

Encapsulation, Inheritance,
Polymorphism, Abstraction

04

Advanced OOP

SOLID principles, Design Patterns,
Architecture



Industry relevance spans
Enterprise Software,
Game Development, GUI
Applications, Web
Frameworks, Mobile
Apps, and System
Design.



What Are Classes & Objects?

Formal Definitions

Class

A **blueprint or template** for creating objects. Defines attributes (data) and methods (behavior) that all objects of that type will possess.

Object

An **instance of a class**. A concrete entity in memory at runtime, with specific values for the attributes defined by its class.

OOP

A programming paradigm based on **"objects"** that contain data (fields/attributes) and code (methods/procedures).

The Blueprint Analogy

Analogy	Class (Blueprint)	Object (Instance)
 House	Architectural design	Actual house built
 Cookie	Cookie cutter	Actual cookies made
 Car	Design specification	Cars on the road
 Bank Account	Account template	Your specific account
 Student	Registration form	Each enrolled student

Key Terminology

Attribute / Field

Data stored in an object

Method

Function that belongs to a class

Constructor

Special method called when object is created

Encapsulation

Bundling data with methods; hiding internal state

Inheritance

Creating new classes based on existing ones

Polymorphism

Same interface, different implementations

Why Do Classes & Objects Exist?

Problems They Solve



Code Duplication

Define behavior once in a class; all objects reuse it automatically



Scattered Data

Group related data and functions together in one logical unit



Complex Systems

Model real-world entities as objects; divide system into manageable pieces



State Management

Each object maintains its own state; methods modify that state safely



Maintainability

Changes isolated to one class; inheritance reduces modification impact

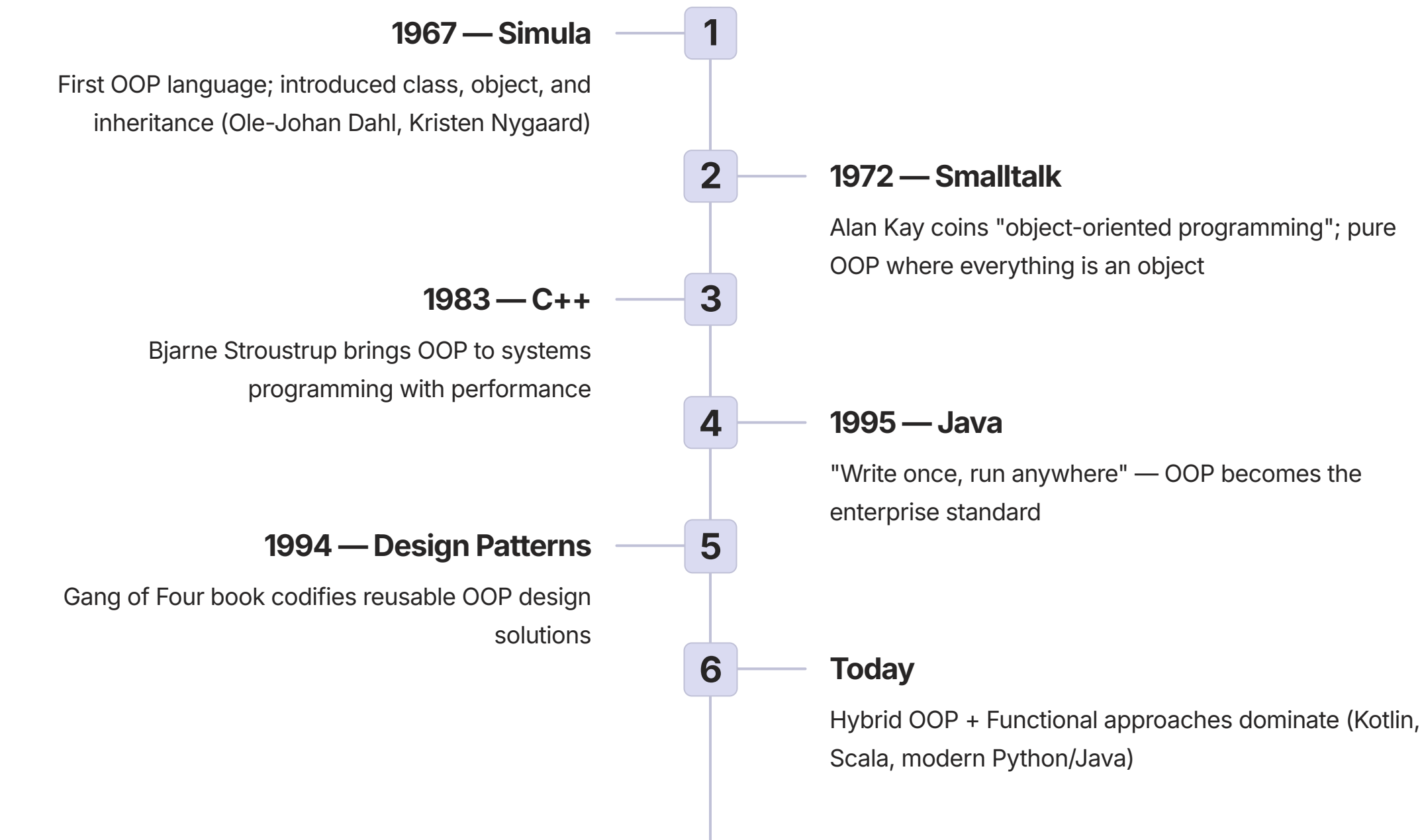


Reusability

Classes reused across projects; inheritance allows extending functionality

 **Without OOP:** A banking system would use separate arrays for account numbers, balances, and owners — adding any new feature requires updating multiple disconnected arrays, causing fragile, unmaintainable code.

Evolution & History of OOP



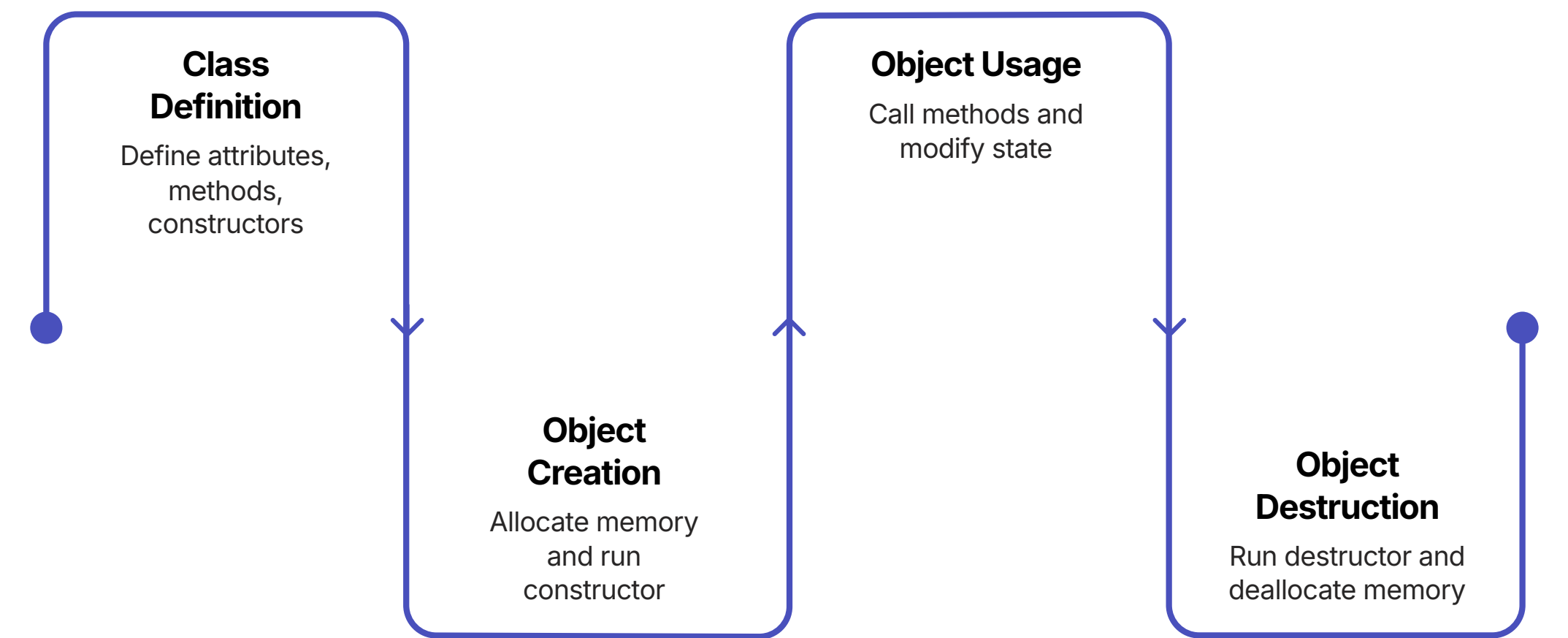
Previous Approaches & Limitations

Era	Approach	Limitation
1950s–60s	Procedural (C, Fortran)	Data and functions separate; global state issues
Pre-OOP	Structs + functions	No data protection; functions operate on multiple types
Monolithic	Single file, all code	No modularity; impossible to scale

Future Trends

- **Hybrid paradigms** — OOP + Functional (Kotlin, Scala)
- **Composition over inheritance** — Go, Rust favor traits/interfaces
- **Immutable objects** — Better concurrency via actor model

How It Works: The Complete OOP Lifecycle



Every object follows this lifecycle — from class definition at compile time through memory allocation, active usage, and final destruction at runtime.

Memory Layout: Stack vs Heap

Stack	Heap
Holds the reference variable (e.g., <code>myCar</code>) pointing to the object's memory address	Holds the actual object data : vtable pointer, field values (model, year), allocated via <code>new</code>

Method Dispatch

- 01

- Follow Reference**
From variable reference, locate object in heap memory
- 02

- Lookup vtable**
Follow vtable pointer to class method table
- 03

- Execute with this**
Call function with implicit `this` = current object address

Types & Variants of Classes

Class Types by Language Paradigm

Classical OOP

Java, C++, C#, Kotlin, Swift

Classes are blueprints; objects are instances; single/multiple inheritance

Prototype-Based

JavaScript (pre-ES6), Lua

Objects inherit directly from other objects; no classical class keyword

Dynamic Classes

Python, Ruby

Classes are objects themselves; open classes allow monkey patching

Struct + Methods

Go, Rust

No classical inheritance; composition over inheritance; interfaces/traits

Constructor Variants

Type	Description	Example
Default	No parameters; initializes defaults	<code>new Car()</code>
Parameterized	Takes arguments to initialize attributes	<code>new Car("Tesla", 2024)</code>
Copy	Creates object as copy of another	<code>new Car(originalCar)</code>
Static Factory	Static method that returns instance	<code>Car.createFromJSON(json)</code>
Private	Constructor cannot be called externally	Singleton pattern

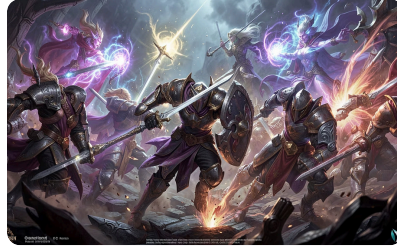
Real-World Use Cases



Banking System

Industry: Banking, Fintech

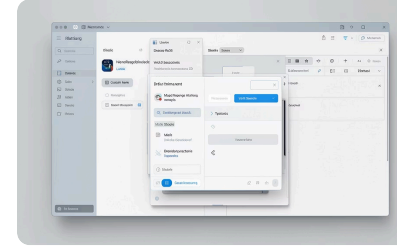
The `BankAccount` class uses **private balance** with public methods that enforce business rules — no negative balances, full transaction logging, and independent state per account.



Video Game Characters

Industry: Gaming (RPG, FPS, MOBA)

Inheritance hierarchy: `Character` → `Warrior`, `Mage`, `Archer`. Polymorphism allows treating all characters uniformly while each has unique behavior.



GUI Framework

Industry: Desktop, Mobile, Web

Class hierarchy: `Widget` → `Button`, `TextField`, `Window`. Each handles its own drawing and events. Millions of apps built on this pattern.

✅ **Key Lesson:** OOP shines when modeling real-world entities — each object maintains independent state, methods enforce business rules, and inheritance reduces code duplication across related types.

Example 1: Basic Class Definition

A language-independent pseudocode example of a `Person` class demonstrating attributes, constructor, instance methods, and static methods.

```
CLASS Person:
  PRIVATE:
    name: STRING
    age: INTEGER
    id: INTEGER
  STATIC next_id: INTEGER = 1000

  CONSTRUCTOR(name: STRING, age: INTEGER):
    this.name = name
    this.age = age
    this.id = Person.next_id
    Person.next_id = Person.next_id + 1

  FUNCTION get_name() -> STRING:
    RETURN this.name

  FUNCTION have_birthday():
    this.age = this.age + 1

  FUNCTION to_string() -> STRING:
    RETURN "Person{name=" + this.name
      + ", age=" + this.age
      + ", id=" + this.id + "}"

  STATIC FUNCTION get_next_id() -> INTEGER:
    RETURN Person.next_id

// Usage:
person1 = Person("Alice", 25)
person2 = Person("Bob", 30)
person1.have_birthday()
PRINT person1.to_string()
// Person{name=Alice, age=26, id=1000}
PRINT person2.to_string()
// Person{name=Bob, age=30, id=1001}
```

What This Demonstrates



Private Fields

`name`, `age`, `id` are hidden from outside — only accessible via methods



Constructor

Initializes all fields when object is created; auto-increments static ID counter



Instance Methods

`have_birthday()` modifies *this* object's state; each object is independent



Static Method

`get_next_id()` belongs to the class, not any instance — called on the class itself

Example 2: Bank Account with Encapsulation

Encapsulation in action — private fields, public methods with validation, and a private helper for logging.

```
CLASS BankAccount:
  PRIVATE:
    account_number: STRING
    owner_name: STRING
    balance: DECIMAL    // private!
    transaction_log: LIST OF STRING

  CONSTRUCTOR(account_number, owner_name,
    initial_deposit):
    this.balance = initial_deposit
    this._log_transaction("Account created")

  FUNCTION get_balance() -> DECIMAL:
    RETURN this.balance

  FUNCTION deposit(amount: DECIMAL) -> BOOLEAN:
    IF amount <= 0:
      PRINT "Must be positive"
      RETURN FALSE
    this.balance = this.balance + amount
    this._log_transaction("Deposited: " + amount)
    RETURN TRUE

  FUNCTION withdraw(amount: DECIMAL) -> BOOLEAN:
    IF amount > this.balance:
      PRINT "Insufficient funds"
      RETURN FALSE
    this.balance = this.balance - amount
    RETURN TRUE

  FUNCTION transfer_to(target, amount) -> BOOLEAN:
    IF this.withdraw(amount):
      target.deposit(amount)
      RETURN TRUE
    RETURN FALSE

  PRIVATE FUNCTION _log_transaction(msg):
    this.transaction_log.append(msg)

// Usage:
acc1 = BankAccount("1001", "Alice", 500.00)
acc2 = BankAccount("1002", "Bob", 300.00)
acc1.deposit(100.00)      // balance = 600
acc1.transfer_to(acc2, 200.00) // acc1=400, acc2=500
// Cannot do: acc1.balance = 9999 (PRIVATE!)
```

Encapsulation Benefits

Data Integrity

Balance can only change through `deposit()` and `withdraw()` — never set directly from outside

Validation

Every method validates inputs before modifying state — prevents negative amounts and overdrafts

Audit Trail

Private `_log_transaction()` automatically records every change — callers can't bypass it

Controlled Access

Read-only `get_balance()` exposes the value without allowing modification

📌 The golden rule: **private fields, public methods**. Never expose internal state directly.

Example 3: Inheritance — Shape Hierarchy

ABSTRACT CLASS Shape:

PROTECTED:

color: STRING

CONSTRUCTOR(color): `this.color = color`

ABSTRACT FUNCTION `area()` -> DOUBLE

ABSTRACT FUNCTION `perimeter()` -> DOUBLE

FUNCTION `get_color()`: RETURN `this.color`

CLASS Rectangle EXTENDS Shape:

PRIVATE: width, height: DOUBLE

CONSTRUCTOR(color, width, height):

SUPER(color)

`this.width = width; this.height = height`

OVERRIDE FUNCTION `area()`:

RETURN `this.width * this.height`

OVERRIDE FUNCTION `perimeter()`:

RETURN `2 * (this.width + this.height)`

CLASS Circle EXTENDS Shape:

PRIVATE: radius: DOUBLE

CONSTRUCTOR(color, radius):

SUPER(color); `this.radius = radius`

OVERRIDE FUNCTION `area()`:

RETURN `PI * this.radius * this.radius`

OVERRIDE FUNCTION `perimeter()`:

RETURN `2 * PI * this.radius`

// POLYMORPHISM IN ACTION:

shapes = [

Rectangle("Red", 5.0, 3.0),

Circle("Blue", 4.0),

Triangle("Green", 3.0, 4.0, 5.0)

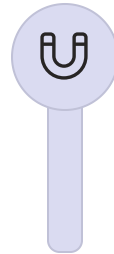
]

FOR each shape IN shapes:

PRINT shape.area() // each calls its OWN area()

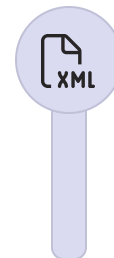
PRINT shape.perimeter() // same interface, diff behavior

Inheritance Concepts Illustrated



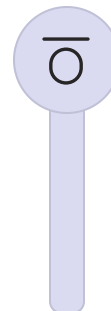
Abstract Class

Shape defines the contract — `area()` and `perimeter()` must be implemented by every subclass



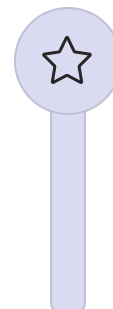
Inheritance (EXTENDS)

Rectangle and Circle inherit color and `get_color()` from Shape — no duplication



Method Override

Each subclass provides its own `area()` formula — same method name, different implementation

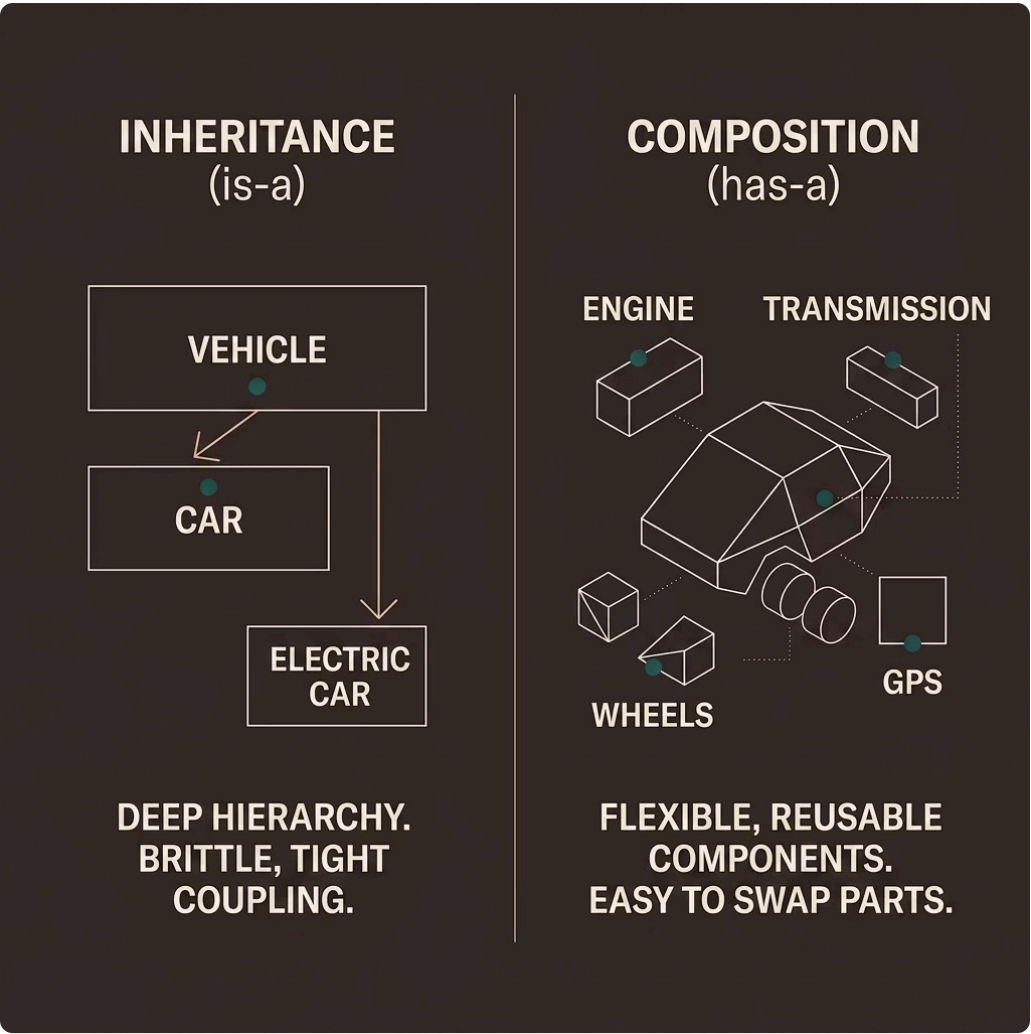


Polymorphism

The loop treats all shapes uniformly — the correct `area()` is called automatically at runtime

Example 4: Composition vs Inheritance

The **Vehicle** example demonstrates the preferred pattern: *composition* (has-a) for reusable parts, and *inheritance* (is-a) only when truly appropriate.



Composition Pattern

Engine

horsepower, type →
start(), stop()

Transmission

gears, type → shift_up()

GPS

→ navigate(destination)

Vehicle uses all three (has-a)

CLASS Vehicle:

```
engine: Engine    // has-a
transmission: Transmission
wheels: Wheels
```

CLASS Car EXTENDS Vehicle: // is-a

```
gps: GPS          // has-a
```

CLASS ElectricCar EXTENDS Car:

```
battery_capacity: INTEGER
```

Rule of thumb: Use inheritance for "is-a" relationships. Use composition for "has-a". When in doubt, prefer composition — it's more flexible.

Lab 1 (Beginner): Student Grade Tracker

Lab Overview

Objective	Create a Student class to track grades
Difficulty	★ Beginner
Prerequisites	Basic class syntax, arrays, methods

Key Methods to Implement

01

add_grade(subject, grade)

Validate grade is 0–100; append to parallel lists

02

get_average()

Sum all grades and divide by count; return 0.0 if empty

03

get_letter_grade()

A≥90, B≥80, C≥70, D≥60, else F

04

get_report_card()

Format all grades + average + letter grade as string

Expected Output

Student: Alice Chen (S1001)

Grades:

Math: 85

Science: 92

English: 78

History: 88

Average: 85.75 (B)

Student: Bob Smith (S1002)

Grades:

Math: 65

Science: 70

Average: 67.5 (D)

✔ **Learning Outcome:** Understand encapsulation (private fields), methods that operate on object state, and object independence — each student object is completely separate.

Lab 2 (Intermediate): Library Management System

Build a system with three collaborating classes: `Book`, `LibraryMember`, and `Library` — demonstrating composition and object references.

Book Class

- Fields: title, author, isbn, is_checked_out, checked_out_by, due_date
- Methods: `check_out(member_id, days)`, `return_book()`, `is_available()`, `get_details()`

LibraryMember Class

- Fields: member_id, name, email, checked_out_books (list of ISBNs)
- Methods: `can_check_out()` (max 5), `add_book(isbn)`, `remove_book(isbn)`

Library Class

- Fields: name, books (dict isbn→Book), members (dict id→Member)
- Methods: `checkout_book()`, `return_book()`, `search_by_title()`, `get_library_status()`

Test Scenario

```
library = Library("City Central Library")
library.add_book(Book("The Hobbit", "Tolkien", "978-..."))
library.add_book(Book("1984", "Orwell", "978-..."))
library.register_member(LibraryMember("M001", "Alice", "..."))

PRINT library.checkout_book("978-...", "M001", 14)
// "Successfully checked out"
PRINT library.checkout_book("978-...", "M002", 14)
// "Book is already checked out"

results = library.search_by_title("Hobbit")
PRINT library.return_book("978-...", "M001")
// "Book returned successfully"
PRINT library.get_library_status()
// "City Central Library: 2/2 books available"
```

✓ **Learning Outcome:** Multiple classes collaborating; object references passed between classes; encapsulation maintained across class boundaries.

Key Design Decisions

- Library uses **dictionaries** for O(1) lookup by ISBN and member ID
- Checkout validates **both** book availability and member limit before proceeding
- Return validates the **correct member** is returning the book they checked out

Advantages & Disadvantages

✓ Advantages

Code Organization

Groups related data and behavior into logical, self-contained units

Reusability

Classes reused across projects via inheritance and composition

Encapsulation

Protects internal state; prevents invalid modifications from outside

Maintainability

Changes isolated to one class; easier debugging and testing

Polymorphism

Write code that works with any derived type — extensible by design

⚠ Disadvantages

Over-Engineering

Classes add unnecessary complexity for simple scripts or small programs

Inheritance Complexity

Deep hierarchies become hard to understand and maintain over time

Learning Curve

Beginners struggle with abstraction, inheritance, and polymorphism concepts

Performance Overhead

Slight overhead vs procedural: vtable indirection, heap allocation, GC

Concurrency Challenges

Shared mutable state in objects requires careful synchronization

Performance Considerations

Complexity of Common Operations

Operation	Time	Notes
Object creation	$O(1)$ – $O(n)$	Allocation + constructor
Field access	$O(1)$	Direct memory access
Non-virtual method call	$O(1)$	Direct function call
Virtual method call	$O(1)$	One extra vtable dereference
Garbage collection	Varies	Language-dependent; can pause

Memory Overhead

Platform	Empty Object Cost
Java / C#	8–16 bytes overhead + fields
C++ with vtable	8 bytes (vptr) + fields
Virtual call overhead	2–3× non-virtual (negligible)

Optimization Techniques



Object Pooling

Reuse frequently created objects to reduce allocation and GC pressure



Value Types (Structs)

Use for small, immutable data — stack allocation, no GC overhead



Final / Non-Virtual

Mark methods as final to prevent overriding and enable inlining

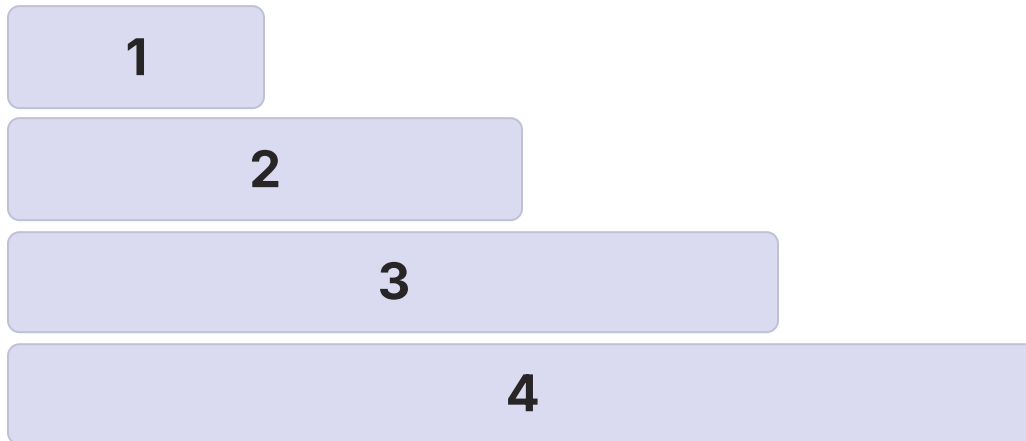


Shallow Hierarchies

Flatter object graphs are easier for compilers to optimize

Scalability & Reliability

Scaling by Codebase Size



1 Small (<10 classes)

Any approach works; simple design is fine

2 Medium (10–100 classes)

Use packages/modules; begin following SOLID principles

3 Large (100–1000+ classes)

Clear architecture required: layers, MVC, design patterns

4 Distributed Systems

Use DTOs for serialization; objects cross network boundaries

Reliability Patterns

Single Responsibility

Each class has only one reason to change

Dependency Inversion

Depend on abstractions (interfaces), not concretions

Factory Pattern

Centralize object creation; easier to modify and test

Immutability

Fields set at construction; never change — inherently thread-safe

RAII (C++) / Dispose (Java/C#)

Constructor acquires resources; destructor/dispose releases them deterministically

Design & Architectural Considerations

Key Architecture Decisions

Decision	Recommendation
Inheritance vs Composition	Prefer composition (more flexible)
Hierarchy depth	Keep shallow (max 3–4 levels)
Access modifiers	Default to private; expose only what's needed
Mutability	Immutable for value objects; mutable for entities
Singleton usage	Use rarely; prefer dependency injection
Abstract vs Interface	Interface for contract; abstract for shared code

OOP Design Patterns

Creational

Factory Method, Abstract Factory, Builder, Singleton, Prototype

Structural

Adapter, Decorator, Facade, Composite, Proxy, Bridge

Behavioral

Observer, Strategy, Command, Template Method, State, Iterator

Maintainability Best Practices



Naming Conventions

Class names: singular nouns (Customer, Order). Methods: verbs (calculateTotal, save)



Unit Testing

Test each class in isolation; use mocks for dependencies to ensure independent verification



Documentation

Document public API; explain preconditions, postconditions, and non-obvious logic

Security Considerations

Threat Model

Threat	Risk
Field injection via reflection	MEDIUM
Exposing internal mutable collections	HIGH
Deserializing malicious data	HIGH
Privilege escalation via subclassing	MEDIUM
Resource exhaustion (object flooding)	MEDIUM

Common Vulnerabilities

Vulnerability	Mitigation
Getter returns mutable list	Return unmodifiable view or copy
Reflection on private fields	Use security manager; avoid reflection
Untrusted deserialization	Validate; use safe deserialization
Sensitive data in object pools	Clear fields before reuse

Hardening Guidelines

Private Fields Always

Never use public fields — prevents external modification of internal state

Defensive Copies

Return copies of mutable objects from getters — protect internal collections

Constructor Validation

Validate all parameters in constructors — prevent invalid object creation

Immutability

Use immutable objects where possible — eliminates state modification risks entirely

Transient Fields

Mark sensitive fields as `transient` (Java) / `NonSerialized` (C#) to exclude from serialization

Best Practices

✓ DO's



Follow SOLID

Single responsibility,
Open-closed, Liskov
substitution, Interface
segregation,
Dependency inversion



Encapsulate Fields

Make fields private;
provide public
getters/setters only when
genuinely needed



Prefer Immutability

Immutable objects for
value types (Point, Color)
— thread-safe by default



Write Unit Tests

Test each class in
isolation; mock
dependencies for clean,
fast tests

✗ DON'Ts

No God Classes

Don't create classes that
do everything — violates
Single Responsibility

No False Inheritance

Don't inherit for code
reuse when "is-a"
relationship is false —
use composition

No Exposed Collections

Don't return internal
mutable lists/arrays
directly from getters

No Overridable Calls in Constructor

Don't call
virtual/overridable
methods in constructors
— subclass not yet
initialized

No Thread-Safety Assumptions

Don't assume objects are thread-safe by default —
synchronize shared mutable state

Common Mistakes & Pitfalls

Beginner Level

Mistake	Why It Happens	Correct Approach
Forgetting new keyword	C/Java background confusion	Always use new for heap objects
Confusing class vs object	Conceptual misunderstanding	Class is blueprint; object is instance
Public fields	Easier to access directly	Use private fields + getters/setters
Using == for object equality	Compares references, not content	Use .equals() (Java) or override __eq__ (Python)

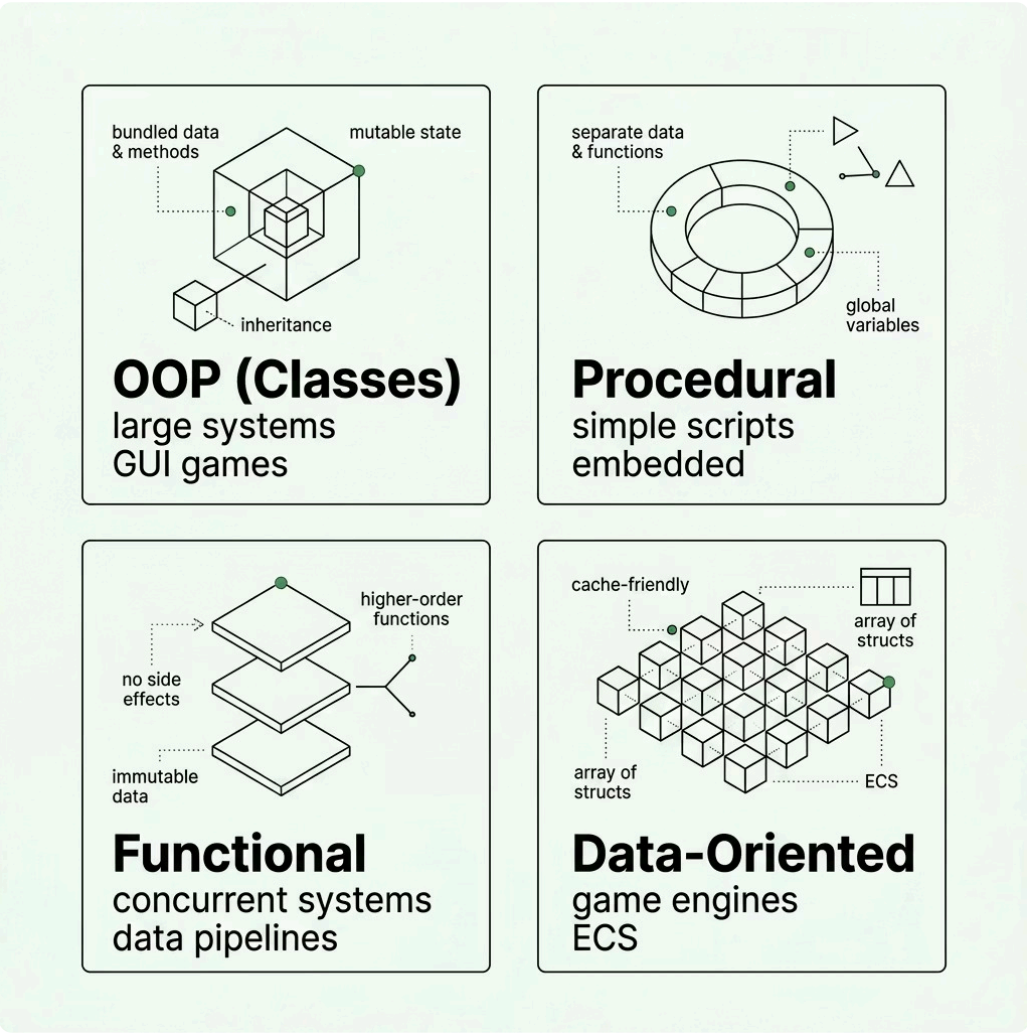
Intermediate Level

Mistake	Impact	Correct Approach
Deep inheritance (5+ levels)	Hard to understand and maintain	Prefer composition; flatten hierarchy
Leaking this in constructor	Inconsistent object state	Avoid; use factory method instead
Not calling super constructor	Parent fields uninitialized	Call super() first in constructor
Mutable objects in sets/maps	Lost in collections after modification	Make objects immutable or don't modify after adding

Advanced / Production Level

Mistake	Impact	Correct Approach
Overusing singleton	Tight coupling; hard to test	Use dependency injection
Ignoring copy constructors	Accidental shared state	Implement proper deep copy/clone
Finalizers causing GC pauses	Resource leaks; performance issues	Use explicit close() or try-with-resources
Serialization without versioning	Deserialization failures on field changes	Use serialVersionUID; versioning strategy

Comparison with Alternatives



Class vs Struct

Criteria	Class (Reference)	Struct (Value)
Memory	Heap	Stack / inline
Assignment	Copies reference	Copies entire data
Nullability	Can be null	Cannot be null (C#)
Inheritance	Full support	Limited / No
GC	Yes	No
Use when	Large objects; need identity	Small, immutable data

Language Feature Comparison

Feature	Java	C++	Python	C#
Multiple inheritance	No (interfaces)	Yes	Yes	No (interfaces)
Destructors	finalize() (deprecated)	~Class()	__del__	~Class()
Properties	Manual	Manual	@property	get/set
Abstract classes	abstract	pure virtual	ABC module	abstract

Learning Summary & Cheat Sheet

Four Pillars of OOP — "A PIE"

A Abstraction

Hide complexity; expose only essential interface to the outside world

P Polymorphism

Same interface, different implementations — runtime dispatch via vtable

I Inheritance

Create hierarchy; reuse code — "is-a" relationship between classes

E Encapsulation

Bundle data + methods; hide internals — private fields, public methods

SOLID Principles — "Sally Orders Large Ice cream Daily"

S — Single Responsibility

One reason to change per class

O — Open/Closed

Open for extension, closed for modification

L — Liskov Substitution

Derived classes can replace base classes

I — Interface Segregation

Don't force unused methods on implementors

D — Dependency Inversion

Depend on abstractions, not concretions

Common Relationships

"is-a" → Inheritance

Car extends Vehicle — Car IS a Vehicle

"has-a" → Composition

Car has Engine — Car HAS an Engine

"uses-a" → Dependency

Driver uses Car — Driver USES a Car

Key Takeaways

Class	Blueprint defining attributes and methods
Object	Concrete instance with specific state
Constructor	Called at birth; initializes the object
Destructor	Called at death; cleans up resources
this/self	Reference to the current object

Memory Aid: "Constructor birth, Destructor death" — and "Private fields, public methods — protect the goods."

Interview Questions & Next Steps

Common Interview Questions



Beginner

- What is the difference between a class and an object?
- What are the four main principles of OOP?
- What is a constructor? When is it called?
- Explain encapsulation with an example.
- What does the `this` keyword refer to?



Intermediate

- Inheritance vs composition — when to use each?
- Explain polymorphism with a real-world example.
- What is the Liskov Substitution Principle?
- How does method overriding differ from overloading?



Advanced

- Explain dynamic vs static dispatch in virtual methods.
- How does the vtable work? Draw memory layout.
- How do you implement immutable objects? Why are they useful?
- Describe the diamond problem in multiple inheritance.

Recommended Next Topics



Inheritance Deep Dive

Virtual methods, hierarchy design, abstract classes vs interfaces



Design Patterns (GoF)

23 reusable OOP solutions to common software design problems



SOLID Principles

Five principles for writing maintainable, extensible OOP code



Dependency Injection

Loose coupling for testability; inversion of control containers



UML Class Diagrams

Visualize class relationships, hierarchies, and system architecture